APPENDIX 4

Ruby Runtimes

Ruby code is converted to executable code using an interpreter. The current default interpreter is called YARV (Yet Another Ruby VM), and has been the standard since Ruby 1.9, replacing the original interpreter, which was known as CRuby or MRI (Matz's Ruby Interpreter). You'll actually still see the names CRuby and MRI used interchangeably with YARV for the current version of the interpreter.

The interpreter makes dozens of choices about how to convert Ruby code to machine code: choices about how to store objects, how to associate objects with their methods, and on and on. Each of these choices has implications for the runtime performance of Ruby.

Not all uses of Ruby are equal. A one-off script could be optimized for quick startup even if that might cause performance issues later—a short script might not have a later. Conversely, a long-running web server may be willing to trade a longer start-up time for better performance later on.

There are now several options for Ruby interpreters. Some, like the Just In Time compilers, are options that ship with core Ruby. Some, like TruffleRuby, are third-party solutions with different speed characteristics. And some, like JRuby, also give Ruby access to other runtime libraries.

Let's go on a tour of the various interpreters available.

Just In Time Compilers

Historically, computer languages are translated in two different ways. A language might use a *compiler* to convert the program code directly to machine language. This compilation happens in a separate step before the code is executed and produces machine-language artifacts. When it's time to run the code, the machine-language version is run, and the original source code is not used.

Ruby typically uses a different tool called an *interpreter*. An interpreter converts the source code to machine language at run time, generally without creating an intermediate machine-language artifact. In an interpreted language, you typically use the original source code at runtime.

That said, the line between compilers and interpreters is blurry. A common technique, used by Java among others, is to compile to a machine-independent byte code and then use a machine-specific runtime interpreter to execute the code. An advantage of this technique is